

Universidad Nacional Autónoma de México



IIMAS



iimas

Licenciatura en Ciencia de Datos

Proyecto final
Procesamiento de lenguaje natural

Fabian Herrera Barron
Emil Ehecatl Sánchez Olsen

9 de junio del 2026

Semestre 2026-1

Introducción

Las proteínas son la base de prácticamente todos los seres vivos, todo lo que ocurre en nuestras células depende de que las proteínas correctas hagan su trabajo. Entender qué hace cada proteína es una de las preguntas más importantes de la biología moderna, en la actualidad se conoce aproximadamente el 5% de las proteínas y sus funciones, este 5% es de constituye millones de proteínas conocidas, describir manualmente la función de cada una es imposible.

La razón por la que se puede aplicar PLN a este problema es la forma en que las proteínas se representan naturalmente. Cada aminoácido se codifica con una letra del alfabeto, por lo que una proteína queda expresada como una secuencia de caracteres, algo que cualquier modelo de lenguaje puede procesar directamente. Esta analogía con el texto no es sólo por la forma en la cual las visualizamos, al igual que en lenguaje natural ciertas palabras tienden a aparecer juntas, en las proteínas ciertos aminoácidos co-ocurren de formas que determinan su estructura y función. Los modelos transformer pueden capturar estas dependencias de largo alcance en la secuencia, lo que los convierte en una herramienta natural para aprender representaciones biológicas ricas sin necesidad de diseñar características manualmente. Estas herramientas son útiles no sólo por su cualidad de poder expresar las propiedades de proteínas que ya conocemos, sino también porque al aprender las propiedades de proteínas conocidas también podrán estimar las propiedades de proteínas nuevas que nunca se han estudiado.

Por esta razón surgen los Modelos de Lenguaje de Proteínas (PLMs). Un PLM aplica exactamente la misma intuición que hay detrás de modelos como BERT (no usa GPT ya que no se desea generar nuevas proteínas), pero en lugar de aprender sobre texto en lenguaje natural, aprende el "idioma" de las proteínas. Pero los PLMs tienen una limitación fundamental, sus salidas son representaciones vectoriales abstractas que no dicen nada en lenguaje humano. El modelo puede capturar propiedades biológicas ricas de una proteína, pero no puede decir qué hace ni por qué importa. Esto no es un problema si sólo se busca hacer análisis sobre las propiedades de estas, pero crea un problema a la hora de interpretar los resultados o intentar interpretar las representaciones vectoriales de cada proteína.

La tarea de proteína a texto busca solucionar esa problemática, conectando un PLM con un modelo de lenguaje natural para que, dada una secuencia de aminoácidos, el sistema genere una descripción en lenguaje humano de las funciones y propiedades de la proteína. Desde la perspectiva de PLN, esto puede verse como un problema de generación condicionada en un escenario multimodal, donde una de las modalidades es biológica en lugar de visual o acústica.

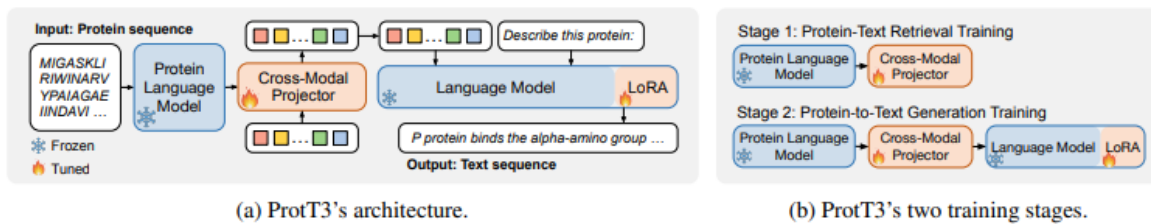
Artículos Seleccionados

ProtT3: Protein-to-Text Generation for Text-based Protein Understanding (Liu et al., ACL 2024)[1]

Los modelos de lenguaje (LMs) son buenos entendiendo descripciones textuales de proteínas, pero fallan cuando se les da directamente una secuencia de aminoácidos por falta de pre entrenamiento en ese tipo de datos. Por otro lado, los modelos de lenguaje para proteínas (PLMs) entienden y representan proteínas, pero no pueden procesar texto. ProtT3 propone resolver esto conectando ambos mundos.

La idea central es usar un proyector cross-modal, específicamente un Q-Former, que actúa como puente entre el espacio de representaciones del PLM y el espacio de entrada del LM. El Q-Former toma los embeddings de la proteína y los comprime en un conjunto fijo de query tokens que el LLM puede interpretar directamente, sin importar qué tan larga sea la secuencia original.

A diferencia de trabajos anteriores que se enfocaban en predicción de propiedades de las proteínas y recuperación proteína-texto, ProtT3 explora el campo de generación de texto a partir de proteínas, estableciendo benchmarks cuantitativos para tareas como protein captioning, protein question-answering y protein-text retrieval.



Este trabajo es la inspiración directa de la arquitectura implementada en este proyecto, adaptando sus componentes principales (ESM-2 como PLM, Q-Former como puente, y un LLM para generación) pero sustituyendo el LLM original por Gemma-2-2B-IT con LoRA y enfocándonos en Q&A.

ProtT3: Protein-to-Text Generation for Text-based Protein Understanding (Liu et al., ACL 2024)[2]

Protein2Text es un modelo de lenguaje multimodal que interpreta secuencias de proteínas y genera texto informativo para responder preguntas abiertas sobre funciones y atributos de proteínas, integrando un mecanismo de resampling dentro de un framework adaptado de LLaVA.

La motivación detrás del trabajo es que el 99.7% de más de 227 millones de secuencias proteicas conocidas siguen sin caracterizar debido a las limitaciones de los métodos experimentales. Protein2Text busca ayudar a los investigadores a acelerar esa caracterización generando descripciones en lenguaje natural directamente desde la secuencia.

Lo relevante de este paper para el proyecto es el dataset que generaron. El dataset Protein2Text-QA está diseñado para generar explicaciones legibles de funciones proteicas a partir de secuencias. Consiste en pares pregunta-respuesta generados a partir de artículos de PubMed Central usando LLaMA 3.1-8B-Instruct, y está dividido en subsets para pre entrenamiento, fine-tuning y evaluación.

El trabajo también señala limitaciones importantes en las métricas de evaluación actuales cuando se aplican a enfoques basados en templates, indicando que pueden llevar a resultados engañosos y enfatizando la necesidad de métodos de evaluación más imparciales. Menciona como ProtT3 y otros modelos se terminan

sobre ajustando a sus datos y sólo funcionan con lenguaje muy formal, ya que el LLM utilizado está especializado en eso.

Metodología

Al tener ambos artículos identificamos que el entrenamiento de protT3 es más robusto, sin embargo tenía varios vacíos, el más importante era el lenguaje utilizado, esto se puede resolver con los datos de Protein2TextQ&A en conjunto con otro LLM, el dataset consiste de 38,585 registros los cuales traen varios campos de interés, como amino_seq y conversations, en la cual está la cadena de aminoácidos limpia y la conversación que se usó para la generación del registro, además para el entrenamiento se tiene question y answer, los cuales son los que se usan para la generación de nuestra respuesta, ya que la pregunta junto con la proteína ya vectorizada son las entradas para nuestro LM, mientras que el answer es el objetivo que usamos para ajustar los pesos del LoRA. El objetivo es construir un modelo que pueda responder preguntas sobre proteínas en lenguaje natural, dada únicamente su secuencia de aminoácidos. Para esto se adoptó una arquitectura multimodal de tres componentes inspirada en ProtT3, donde cada módulo cumple una función específica: extraer información de la proteína, comprimir esa información, y generar texto coherente.

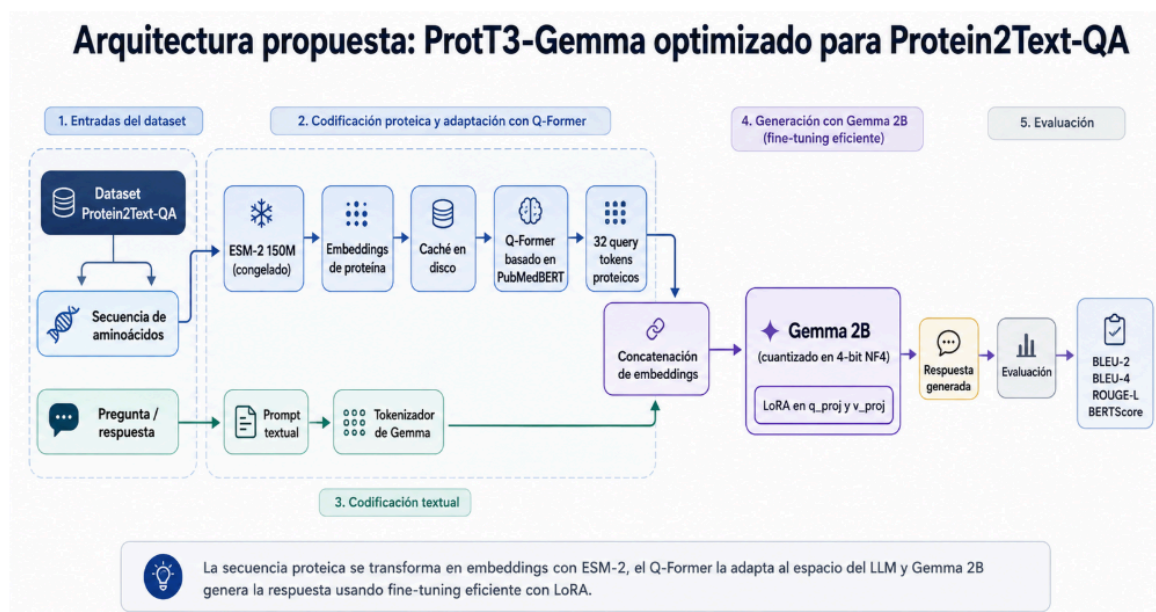
El primer componente es ESM-2 (esm2_t30_150M), un modelo de lenguaje para proteínas pre entrenado de Facebook que toma la secuencia de aminoácidos y produce embeddings. Este módulo se mantiene completamente congelado durante el entrenamiento, ya que sus representaciones vectoriales ya tienen la información estructural y funcional de las proteínas.

El segundo componente es un Q-Former (Querying Transformer) basado en PubMedBERT, que actúa como puente entre el espacio vectorial de las proteínas y el espacio textual. El Q-Former recibe los embeddings de ESM-2 vía cross-attention y los comprime en un conjunto fijo de 32 query tokens, sin importar qué tan larga sea la secuencia original. Estos tokens son los que se le pasan al LLM, no los embeddings crudos.

El tercer componente es Gemma-2-2B-IT, el modelo de lenguaje que genera la respuesta final. Para hacerlo eficiente en memoria se cargó en cuantización 4-bit (NF4) y se adaptó con LoRA con rango 8, modificando sólo las proyecciones de queries y valores de la atención.

Durante el forward, los 32 query tokens del Q-Former se concatenan como prefijo a los embeddings del prompt textual, y el LLM genera la respuesta con ayuda de ambas fuentes de información.

La arquitectura final fue la siguiente:



Implementación

Datos: Se usó el dataset Protein2Text-QA de TumorAI Lab. Se tomó el 30% del split de test, que luego se dividió 95/5 en train y validación. Cada muestra contiene una secuencia de aminoácidos, una pregunta y una respuesta esperada en lenguaje natural. Para evitar data leakage las secuencias que estaban en el test no fueron vistas durante el entrenamiento.

Preprocesamiento: Los embeddings de ESM-2 se precomputaron y cachearon en disco usando un hash SHA-1 de la secuencia como nombre de archivo, para evitar calcularlos en cada época. Las secuencias se truncaron a 1024 y los textos a 768 tokens.

Q-Former: Se cargó PubMedBERT con `add_cross_attention=True`, lo que agrega capas de cross-attention que parten inicializadas aleatoriamente. Se agregó una proyección lineal de 640→768 para alinear la dimensión de ESM-2 con el hidden size de PubMedBERT, y otra proyección de salida de 768→2304 para conectar con Gemma.

Entrenamiento: Se usó AdamW con learning rates diferenciados: 1e-4 para el Q-Former y 1e-5 para LoRA, con weight decay de 0.01 y scheduler coseno con 3% de warmup. El batch efectivo fue de 16 (`batch_size=2`, `gradient accumulation=8`). Se entrenó por 2 épocas con la precision en bfloat16.

Evaluación: Se midió BLEU, ROUGE-1, ROUGE-2, ROUGE-L y BERTScore (F1 con `distilbert-base-uncased`) sobre el conjunto de validación, generando respuestas con greedy decoding y máximo 128 tokens nuevos.

Resultados

Implementación para colab

La primera implementación que fue optimizada para colab con las RAMs de 16GB, en la cual se redujo la precisión de los números, el número de épocas, las capas del LoRA y el tamaño del batch obtuvo un valor muy regular obteniendo los siguientes scores usando las métricas de BERTScore

Precision	0.6385
Recall	0.7757
F1	0.6998

Al ver las respuestas notamos que no se generaban bien los resultados esperados y se tenían respuestas del tipo

Respuesta del modelo:

Yes, it is.

Respuesta esperada:

Yes, it plays a crucial role in iron absorption..

Implementación en Servidor

Con el objetivo de mejorar los resultados obtenidos en la primera versión, se realizó una segunda implementación aprovechando un entorno con más poder de cómputo. Los principales cambios se enfocaron en tres áreas: cantidad de datos, capacidad del entrenamiento y duración de este. Ya que no tenemos la restricción de tiempo ni de poder de GPU que teníamos en colab.

Datos: Se aumentó la proporción del split de test utilizado del 10% al 30%. Esto resultó en aproximadamente 5700 muestras para entrenamiento y 300 para validación.

Batch size: Se duplicó el batch size de 2 a 4, lo que junto con la reducción del gradient accumulation de 8 a 4 mantuvo un batch efectivo de 16, pero esto permite que cada paso de forward procesara más muestras y fuera más estable numéricamente. Esto no es posible sin una GPU de más de 16 GB.

Épocas: Se aumentó el número de épocas de 2 a 5, dando más tiempo al Q-Former para aprender a comprimir correctamente los embeddings de ESM-2 y al adaptador LoRA para ajustar el comportamiento de Gemma para las respuestas.

Scheduler: Se ajustó el warmup de 3% a 5% de los pasos totales, dando al optimizador más tiempo para estabilizarse antes de que el learning rate comience a bajar.

El resto de la arquitectura se mantuvo igual: ESM-2 congelado como encoder proteico, Q-Former basado en PubMedBERT con 32 query tokens, y

Gemma-2-2B-IT en cuantización 4-bit NF4 con LoRA de rango 8 sobre q_proj y v_proj.

Con esta nueva arquitectura se pudo ejecutar en un servidor con mayor poder computacional y los resultados obtenidos con el conjunto de test usando las métricas de BERTScore fueron los siguientes:

Precision	0.8585
Recall	0.8432
F1	0.8496

Con esta segunda implementación se observó una mejora considerable respecto a la versión anterior. Mientras que en Colab se obtuvo un BERTScore F1 de 0.6998, en el servidor se alcanzó un F1 de 0.8496, lo que representa una mejora de casi 15 por ciento. De igual forma, la precisión subió de 0.6385 a 0.8585 y el recall de 0.7757 a 0.8432, indicando que el modelo no solo genera respuestas más parecidas semánticamente a las esperadas, sino que también cubre mejor el contenido relevante de las respuestas.

Para ver esto nuevamente vimos las respuestas que generaba este modelo, un ejemplo es el siguiente:

Pregunta: Is the protein capable of inhibiting other substances?

Predcción: Yes, the protein is capable of inhibiting other substances, such as the enzyme 5-lipoxygenase.

Verdad: Yes, the protein can act as an inhibitor for other substances.

Esta mejora se atribuye principalmente al aumento en la cantidad de datos de entrenamiento, el mayor número de épocas y el tamaño del batch, lo que permitió que el Q-Former convergiera de una mejor forma y que el adaptador LoRA tuviera suficiente señal para ajustar el comportamiento de Gemma hacia la tarea específica de respuesta sobre proteínas.

Discusión

Debido a la falta de poder computacional no fue posible replicar un entrenamiento comparable al de ProtT3, ya que en dicho artículo se utilizaron dos GPUs A100-40GB y cuatro GPUs V100-32GB. En contraste, las primeras semanas del proyecto se trabajó con GPUs de 16GB en Google Colab, lo que obligó a reducir considerablemente varios hiperparámetros como el menor número de épocas, batch size reducido, menos capas de LoRA y precisión reducida a 4-bit para que los cálculos cupieran en memoria, además de un límite de 4 horas de uso de GPU. Con estas condiciones, la primera implementación obtuvo resultados regulares, con un BERTScore F1 de 0.6998, y al revisar las respuestas generadas de forma manual se observó que el modelo producía salidas muy cortas y sin contenido útil, como responder "Yes, it is." cuando la respuesta esperada era "Yes, it plays a crucial role in iron absorption ..."

Durante las últimas semanas del proyecto se obtuvo acceso a un servidor con dos NVIDIA RTX A5000 de 24GB cada una, lo que permitió relajar varias de las restricciones anteriores. Al poder distribuir los cálculos entre ambas GPUs sin preocuparse tanto por el uso de memoria, fue posible aumentar el porcentaje de datos utilizados del 10% al 30% y subir el número de épocas de 2 a 5 y aumentar el batch size. Estos cambios tuvieron un impacto directo y notable en los resultados, subiendo el BERTScore F1 de 0.6998 a 0.8496, con mejoras consistentes también en precisión (0.6385 $\rightarrow$ 0.8585) y recall (0.7757 $\rightarrow$ 0.8432).

BERTScore	Colab GPU de 16GB	Servidor 2 GPUs de 24GB
Precision	0.6385	0.8585
Recall	0.7757	0.8432
F1	0.6998	0.8496

Aunque los resultados mejoraron bastante, aún existe una diferencia respecto a lo reportado en ProtT3, lo cual es esperable ya que ese trabajo utilizó

significativamente más recursos computacionales, un pre entrenamiento dedicado del Q-Former y un dataset de mayor escala. Aun así los resultados obtenidos demuestran que la arquitectura propuesta es viable y que con mayor tiempo de entrenamiento y más datos los scores podrían seguir mejorando.

Para las posibles mejoras notamos que nuestro modelo cubre sólo uno de los tres principales problemas, pero todavía no se solucionan los otros dos: no tomar en cuenta la estructura 3D de las proteínas y el problema de las posibles alucinaciones de los LM. Para las estructuras 3D se menciona que es necesario un poder de cómputo aún mayor ya que con el que se usó para entrenar protT3 (que es mucho mayor al nuestro) tardarían aproximadamente 4 meses en obtener los resultados, por lo que esto no es posible. El otro punto sobre las alucinaciones podría corroborarse con una base de datos como PubMed o Go, sin embargo una de los principales objetivos de esto es que se puedan estimar propiedades de proteínas que no se han estudiado previamente, por lo cual sus características no podrían ser encontradas en estas bases de conocimiento.

Código

Para reproducibilidad tenemos el código abierto en el siguiente repositorio https://github.com/FabianHeBa/Proyecto_PLN_ProTT3/tree/main/prott3_p2t

Referencias

- [1] Z. Liu, A. Zhang, H. Fei, E. Zhang, X. Wang, K. Kawaguchi, and T.-S. Chua, "ProtT3: Protein-to-Text Generation for Text-based Protein Understanding," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Bangkok, Thailand, Aug. 2024, pp. 5949–5966.
- [2] A. Jararweh, O. Macaulay, D. Arredondo, Y. Hu, L. E. Tafoya, K. Virupakshappa, and A. Sahu, "Protein2Text: Resampling Mechanism to Translate Protein Sequences into Human-Interpretable Text," in Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics:

Human Language Technologies (Volume 3: Industry Track), Albuquerque, New Mexico, Apr. 2025, pp. 918–937.